

# Manual del Proyecto

ArgentumOnline - Documentación Interna y Reporte de Desarrollo

## 1. Estructura del Equipo y Asignación de Roles

El desarrollo de la recreación de Argentum Online fue abordado de manera modular, distribuyendo las responsabilidades principales según los subsistemas del juego (Cliente, Servidor, Editor de Mapas y Protocolo de Comunicación). A continuación, se detalla la composición del equipo, el entorno de desarrollo adoptado por cada integrante y sus tareas específicas:

| Integrante / Legajo                       | Área Principal                     | Responsabilidades Técnicas                                                                                                                                                                                                                | IDE Base |
|-------------------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Maurizio Reina</b><br>108119           | Desarrollo del Cliente             | Diseño e implementación de la interfaz gráfica y la capa visual del juego.<br>Renderizado del mapa en tiempo real y sincronización de las animaciones del jugador.                                                                        | IntelliJ |
| <b>Jose Ignacio Adelardi</b><br>111701    | Menus de UI y Multimedia           | Desarrollo de los menús dinámicos utilizando el framework Qt5. Integración del sistema multimedia, incluyendo la reproducción de música incidental y efectos de sonido en base a los eventos del juego. Soporte general a la capa visual. | VS Code  |
| <b>Camila Miranda Vandevale</b><br>112777 | Lógica del Servidor y Persistencia | Implementación del loop lógico del servidor, persistencia de datos de los personajes, inteligencia artificial básica de los enemigos (NPCs hostiles) y resolución de mecánicas de combate y economía del juego.                           | VS Code  |
| <b>José Tissera</b><br>112788             | Protocolo y Herramientas           | Diseño del protocolo de red para la comunicación Cliente-Servidor.<br>Colaboración en la lógica interna del servidor, desarrollo del Editor de Mapas dinámico en Qt5 y las herramientas de construcción de escenarios.                    | VS Code  |

## 2. Organización Semanal y Desviación del Plan Inicial

El proyecto se estructuró de forma iterativa, priorizando en las primeras semanas la conectividad básica y la arquitectura del motor, para luego avanzar hacia las features de gameplay y el pulido de la interfaz.

Sin embargo, la complejidad inherente generó ciertas desviaciones respecto a los objetivos ideales inicialmente planteados.

### Funcionalidades No Implementadas por Restricciones de Tiempo:

- **Persistencia de ítems en el suelo:** Inicialmente se planificó que los objetos que los jugadores tirasen o dejaran caer en el mapa se conservaran tras el cierre o reinicio del servidor. Debido a la prioridad otorgada a la estabilidad de las funcionalidades principales, esta persistencia quedó fuera del alcance de la entrega.
- **Persistencia de los NPC banqueros:** Los banqueros resguardan el oro e ítems, pero el inventario de objetos depositados en el banco por cada jugador no llegó a persistirse de forma definitiva en el almacenamiento secundario al apagar el entorno del servidor, manteniéndose únicamente de forma volátil en memoria durante la ejecución de la sesión.

### Documentación clave para la optimización del Gameloop

Para la construcción de un bucle de juego dinámico y eficiente que evitara desincronizaciones de FPS entre el cliente y el servidor, el equipo utilizó como pilar teórico la documentación provista por la cátedra sobre arquitecturas de simulación de tiempo constante:

<https://book-of-gehn.github.io/articles/2019/10/23/Constant-Rate-Loop.html>

## 3. Desafíos Técnicos y Puntos Críticos

A lo largo del ciclo de vida del desarrollo, el equipo se enfrentó a dos grandes problemáticas arquitectónicas que requirieron un esfuerzo considerable de diseño:

### A. Escalabilidad del Protocolo de Comunicación

Al comenzar el desarrollo, el volumen de mensajes e interacciones entre el Cliente y el Servidor escaló de forma exponencial. Para mitigar la complejidad y evitar un código acoplado y difícil de mantener, se invirtió un esfuerzo crítico inicial en estructurar el protocolo bajo el **Principio de Open/Close (OCP)**. Esto permitió que la adición de nuevos comandos multimedia y de movimiento no requiriera la modificación estructural del despachador central de red, garantizando la sostenibilidad a largo plazo.

### B. El Sistema de Colisiones basado en Tiles

El punto más problemático del proyecto radica en el motor de colisiones. El sistema se estructuró de manera rígida midiendo las posiciones en "tiles" estáticos de 32x32 píxeles. En la práctica, esto generó graves desajustes visuales debido a que los recursos gráficos (sprites) del juego no siempre eran cuadrados ni múltiplos exactos de 32.

Como consecuencia, persisten errores conocidos donde ciertos elementos del mapa se pueden "atravesar" visualmente de manera parcial, o bien, el personaje colisiona de forma invisible a una distancia notable del objeto real. A pesar de los esfuerzos de pulido en la geometría lógica de los assets,

una resolución integral de este problema requeriría replantear por completo la arquitectura de colisiones hacia un modelo de bounding boxes continuas (ejes flotantes) en lugar de celdas discretas.

## 4. Lecciones Aprendidas y Retrospectiva

Si el equipo tuviera la oportunidad de reiniciar el desarrollo del proyecto desde cero, se aplicarían modificaciones esenciales en dos niveles fundamentales:

- **A Nivel Código:** Se descartaría el diseño de colisiones basado en lógica de celdas unitarias desde el día uno, optando en su lugar por un motor de físicas simplificado que asocie cajas de colisión personalizadas a cada sprite de manera independiente a su tamaño.
- **A Nivel Organizacional:** Se incrementaría la frecuencia de las pruebas de integración cruzadas entre la UI de Qt5, el cliente SDL y el Servidor lógico en etapas tempranas. Desarrollar la lógica de escenarios y el protocolo de forma aislada generó problemas durante las fases de integración.